

Structuri de Date si Algoritmi

Examen (RESTANTA)

22 iunie 2022

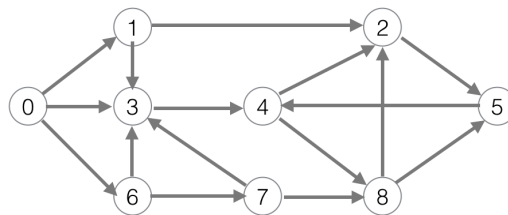
Timp de lucru: 120 min

Nume student _____

Grupa _____

Întrebare	1	2	3	4	5	6	7	8	9	10	11	12	13	Total
Punctaj	10	10	10	10	10	10	10	10	10	10	20	20	20	160
Scor														

- (10 puncte) Se da o tabela de dispersie (T) de dimensiune $m = 13$, ce utilizeaza adresare deschisa si verificare liniara, si functia de dispersie: $h(key, i) = (key \% m + i) \% m$.
 - Inserati, succesiv, in tabela, urmatoarele chei: 11, 3, 16, 24, 50, 76, 104. Desenati cum arata tabela de dispersie la finalul succesiunii de inserari. Care este valoarea factorului de umplere dupa inserari?
 - Care este valoarea la care ajunge i la executia fiecareia din urmatoarele operatii (dupa inserari): Hash-search(T, 50), Hash-search(T, 100), respectiv Hash-search(T, 19).
 - Dati un exemplu de cheie care nu se mai poate insera in tabela dupa inserarile efectuate, sau motivati de ce nu se poate gasi o asemenea cheie.
- (10 puncte) Se da problema selectiei activitatilor, si doi algoritmi greedy pentru aceasta problema:
 - unul care foloseste ca euristica durata unei activitati – *MinDurationActivitySelector*(A, s, f)
 - celalalt care foloseste ca euristica timpul de finalizare – *EarlyFinishActivitySelector*(A, s, f)
 Pentru activitatile $A = \{p, q, r, s, t, u, v, w, x, y, z\}$ cu urmasorii timpi de incepere si finalizare: $s = \{1, 3, 0, 5, 3, 5, 6, 8, 8, 2, 12\}$, respectiv $f = \{4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14\}$, ce activitati vor fi selectate in urma aplicarii fiecaruia dintre acesti 2 algoritmi? E vreuna din solutii optima? Motivati.
- (10 puncte) Se da graful de mai jos.
 - Aplicati algoritmul *BFS*(1). Desenati arborele rezultat.
 - Desenati continutul cozii la fiecare pas al algoritmului de la punctul (b).



- (10 puncte) Pe graful de la problema 3:
 - Desenati arborele rezultat in urma aplicarii algoritmului *DFS_Visit*(1).
 - Marcati, pe arbore, timpii de descoperire si de finalizare ai fiecarui nod parcurs.
 - Macati, pe graf, tipul fiecarei muchii, asa cum reiese din parcurgerea ceruta.
- (10 puncte) Se da urmatoarea secventa de intregi: 9, 6, 2, 14, 18, 5, 3, 11.
 - Construiti un arbore binar de cautare, inserand, pe rand, cheile date. Desenati arborele.
 - Dati secventa de parcurgere in pre-ordine a arborelui. Care este valoarea inaltimii arborelui?
 - Desenati arborele, dupa fiecare din operatiile succesive: Tree-Delete(T, 9), Tree-Delete(T, 6), Tree-Insert(T, 6).
- (10 puncte) Se da urmatorul sir de dimensiune 7: {9, 18, 2, 6, 14, 3, 5} si algoritmul QuickSort:

```

QUICK-SORT(A, low, high)
    if low < high
        then q = PARTITION(A, low, high)
        printArray(A, low, high)
        QUICK-SORT(A, low, q-1)
        QUICK-SORT(A, q+1, high)
        printArray(A, low, high)

```

Presupunand ca functia PARTITION utilizeaza ca pivot elementul de pe ultima pozitie din sub-sirul curent, iar printArray afiseaza continutul unui sir pe o linie, ce se va afisa pe ecran in urma apelului QUICK-SORT(A, 1, 7)?

7. (10 puncte) Pentru fiecare din algoritmii descrisi mai jos in pseudocod: (1) calculati numarul de apeluri ai functiei op(), in functie de N si (2) estimati complexitatea algoritmului

- | | |
|--|--|
| <p>(a) f1(int N)
 for i = 1 to N
 op()</p> | <p>(b) f2(int N)
 for i = N downto 1 by i=i/2
 f1(i)</p> |
| <p>(c) f3(int N)
 if N <= 1 then
 op()
 else
 f3(N/2)</p> | <p>(d) f4(int N)
 if N <= 0 then op()
 else f4(N-1) + f4(N-1)</p> |

8. (10 puncte) Se da problema gasirii sub-secventei comune de lungime maxima (Longest Common Subsequence). Pentru solutia care utilizeaza tehnica programarii dinamice, se cere sa calculati continutul matricilor *b* si *c* rezultate in urma aplicarii algoritmului LCS-Length(X,Y), pentru sirurile X = sdais si Y = magic.
9. (10 puncte) Considerati mesaje care contin doar vocale (A, E, I, O, U), cu urmatoarele probabilitati de aparitie: A: 0.22, E: 0.34, I: 0.17, O: 0.19 si U: 0.08. Folosind algoritmul lui Huffman, construiti codificarea de lungime variabila, presupunand ca fiecare vocala este codificata separat. Desenati arborele rezultat si dat codificarea pentru fiecare vocala. Calculati lungimea asteptata a unui mesaj continand 100 de vocale.
10. (10 puncte) Comparati si contrastati tehnica Backtracking cu Programarea Dinamica (strategie, complexitate, memorie aditionala, aplicabilitate, calitatea solutiilor gasite, etc).